

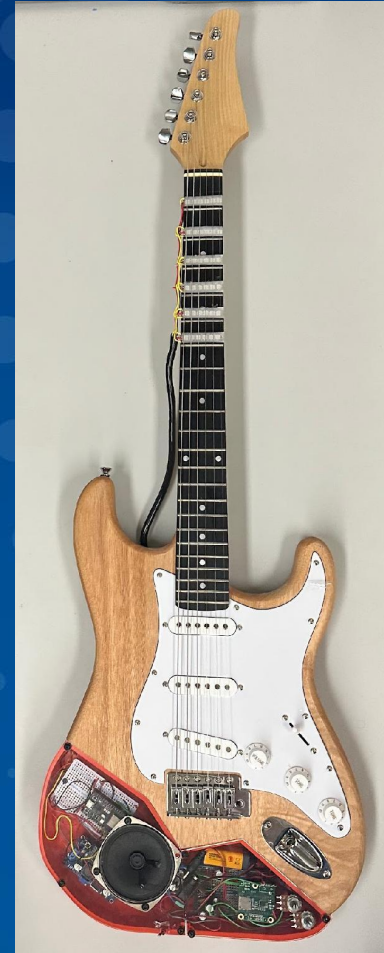
StrumTech; Innovative Guitar System for Beginners

David Ellis; Digital Effects, Circuitry and Guitar Body

Austin Petersen; PCB, 3D Print Designs and Guitar Body/Neck

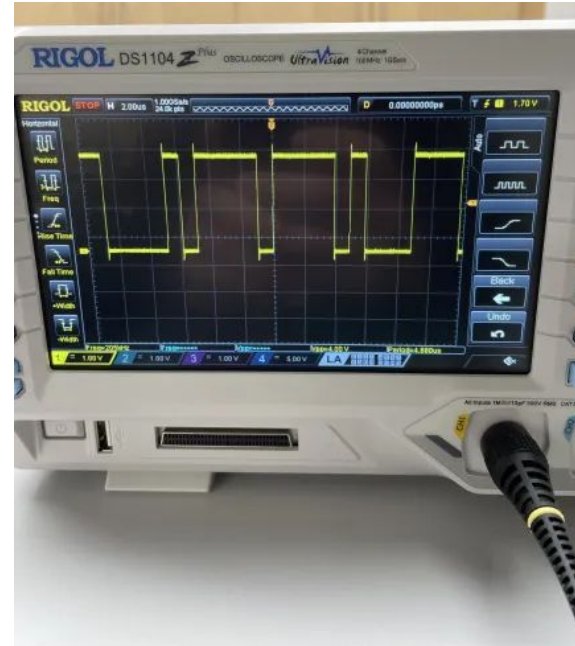
Gerald Stanford; App, Coding and Testing

Michael Ward; Coding, Circuitry and Guitar Body/Neck



Engineering Standards and Codes

- ISO/IEC 14882; C++ Coding Standard
- ISO 16:1975; A440 Hz Standard
- I2S Communication Specification
- Bluetooth BLE Specification
- JSON ECMA-404 Specification



Analog Input
Signal



I2S



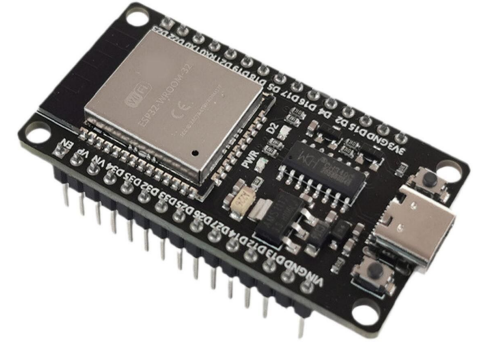
I2S



Analog Output
Signal

Constraints

- Size of Guitar
- Cost \$
- Processing Power
 - Light array
 - DSP
 - Tuner
 - Bluetooth
- Speed/Latency
- Time
- Apple dev Ecosystem
 - Cost
 - Testability



**\$250
Budget**

State of Project from Fall

<u>Completed</u>	<u>To be Completed</u>
Code for Tuner	Code for Digital Effects
Code for LED Fretboard	Guitar Body
Amplifier Circuit	Finalized LED Fretboard
Basic LED Fretboard	Bluetooth Connectivity
Start of App	Finalized App



Initial Game-plan for the Spring

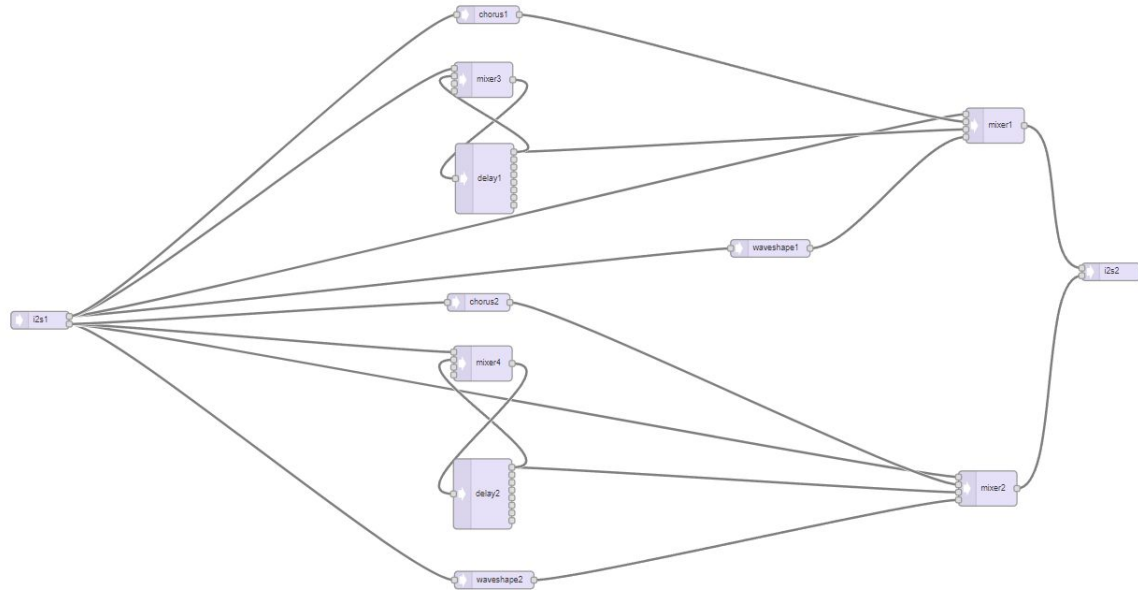
For the spring semester, our goals were to complete the app and insert the physical components into the guitar. Our specific goals were as follows:

- Complete Phone App, including
 - Tuner Interface
 - Digital Pedal Interface
 - Chord Interface
- Change PCB design to repair connection issues
- Build Guitar and Insert Components into Guitar and Fretboard
- Complete Digital Effects component and receive proper sound from it



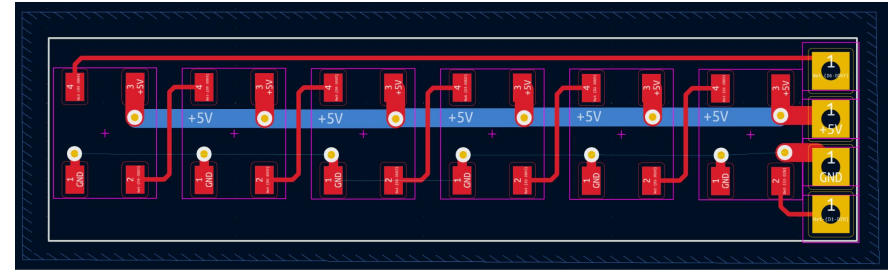
Implementation of Digital Effects

- Teensy audio adapter board and audio library
- Built in ADC and DAC
- Attempted to implement delay, chorus, and distortion
- Resulting effects were delay and “techno-distortion”



New PCB and Construction of the LED Fretboard

- Due to connection errors in our Fall PCB design, we changed our PCB to have through-hole soldering.
- We cut channels into the fretboard to allow the insertion of the PCB into the neck.
- We 3D printed translucent covers for the LEDs.
- We then epoxied the PCBs onto the fretboard and hot glued the translucent covers onto them.



New PCB Design with Through-Hole

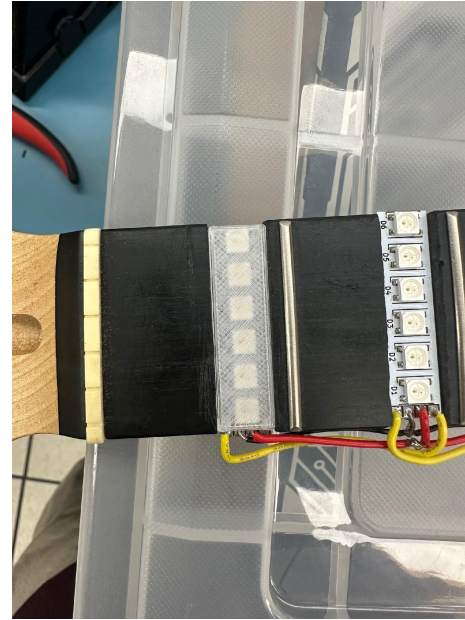
New PCB and Construction of the LED Fretboard



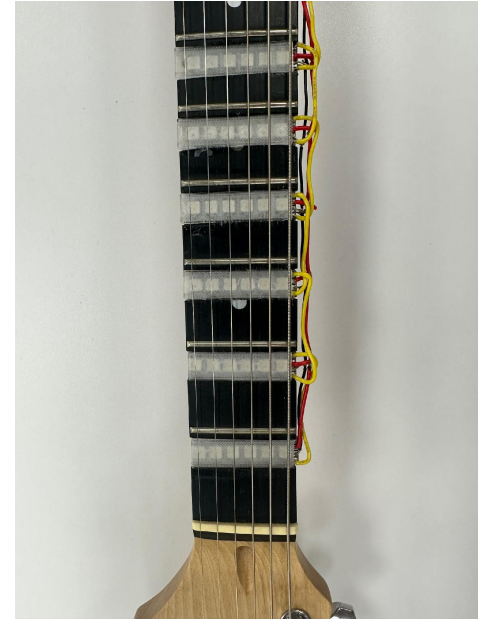
Cutting Channels in the
Fretboard



Preliminary LED Strip
Placement and Soldering



Translucent Cover for the
LED Strips



Finalized LED Fretboard

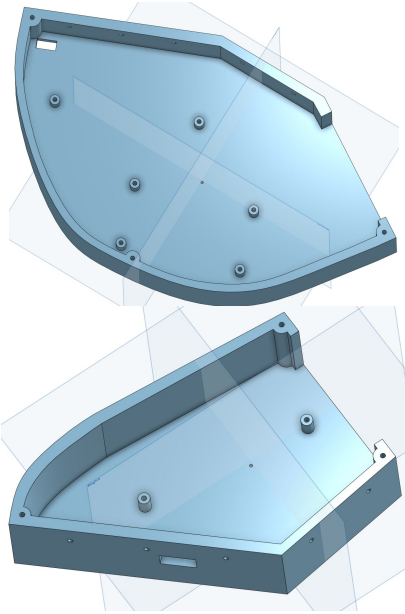
Design of Guitar Body

- After receiving our guitar kit, we determined that there wouldn't be enough space within the wood to insert all of our electronics.
- We opted for designing a 3D print cavity and cutting off part of the guitar to insert this cavity.
- We also decided to have any acrylic cover of the cavity to allow the viewing of all of the internal components.

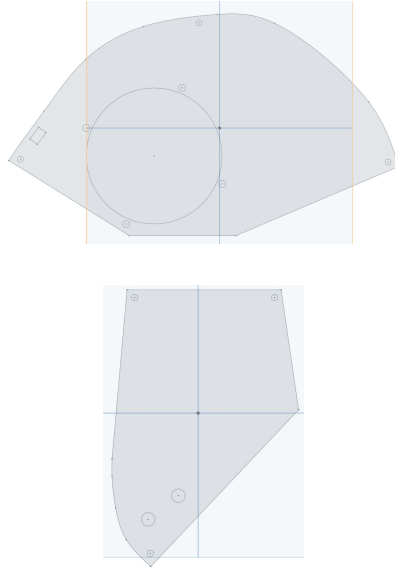


Initial Guitar Body from Guitar Kit

Design of Guitar Body



CAD Design for the Guitar Cavity



CAD Design for Acrylic Cover



Guitar Body with Cavity Space Cutoff



Finalized Guitar Body with 3D Printed Cavity

App Coding and Design

State Machine of the MicroController.

```
/* determines which mode to switch to and handles the switching logic*/
void ExecuteCurrentMode()
{
    Serial.println("Current Mode: ");
    Serial.println((int)CURRENT_MODE_SETTINGS);
    NEXT_MODE_SETTINGS = CheckIfBluetoothRecievedModeData(CURRENT_MODE_SETTINGS);

    int nextMode = NEXT_MODE_SETTINGS;
    int currentMode = CURRENT_MODE_SETTINGS;

    if(nextMode == currentMode && currentMode != NO_MODE)
    {
        GUITAR_STATE_PROGRESS = GUITAR_STATE_CONT;
        SwitchToMode(currentMode);
    }
    else if (nextMode != currentMode && currentMode != NO_MODE && nextMode != NO_MODE)
    {
        GUITAR_STATE_PROGRESS = GUITAR_STATE_EXIT;
        SwitchToMode(currentMode);

        GUITAR_STATE_PROGRESS = GUITAR_STATE_NEW;
        NewModeInit(nextMode);
        SwitchToMode(nextMode);
    }
    else if (currentMode == NO_MODE && nextMode != NO_MODE )
    {
        GUITAR_STATE_PROGRESS = GUITAR_STATE_NEW;
        NewModeInit(nextMode);
        SwitchToMode(nextMode);
    }
    else if (nextMode == NO_MODE)
    {
        // nextMode == NO_MODE DO NOTHING just delay
    }
}
```

App Coding and Design

MICROCONTROLLER:

State machine

- Switches through the different modes: Free play, Tuner, Chords
- Each mode has 3 states: New, Cont'd, exit
- Different capabilities of the learning guitar system are brought together here

Bluetooth configuration data

- Services - encompasses groups of data (organization)
- Characteristics - different data sent

Bluetooth communications / advertising



App Coding and Design

APP:

User Interface:

- Chord selection Screen
- Tuner Screen
- Digital Effects Screen

Bluetooth Communication:

- Send data to the controller to alter state and Receive data to display

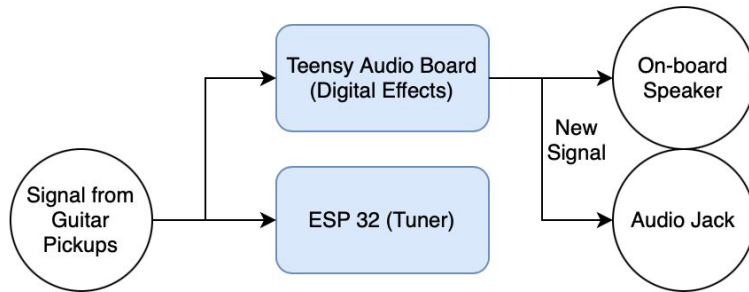
Mode Control:

- Change between the different guitar modes available on the controller by navigating between the different screens

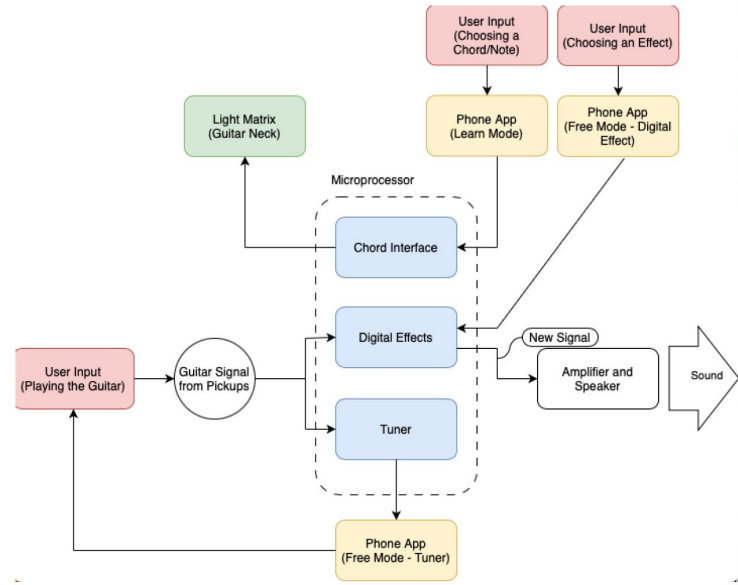


Circuit and Block Diagram

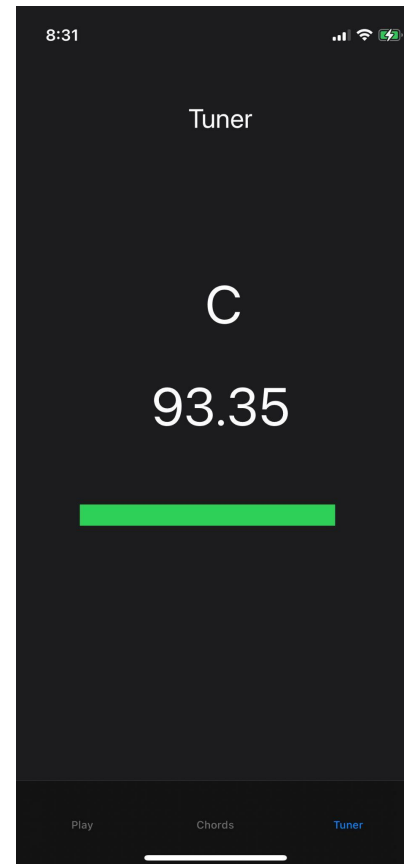
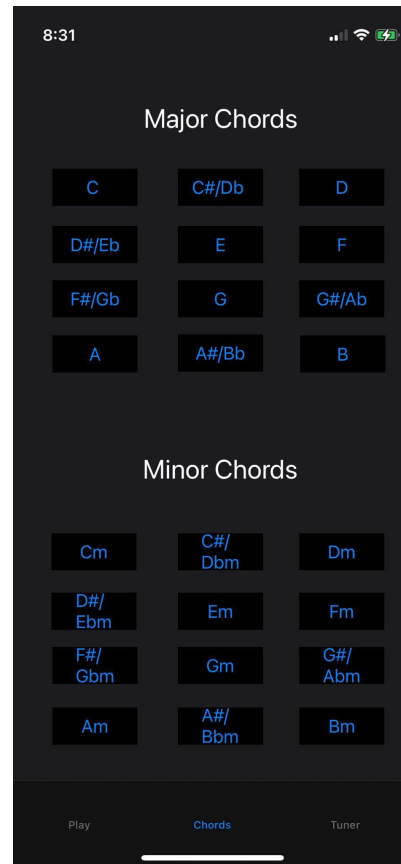
Audio Signal Circuit Diagram



System Block Diagram



Demonstration



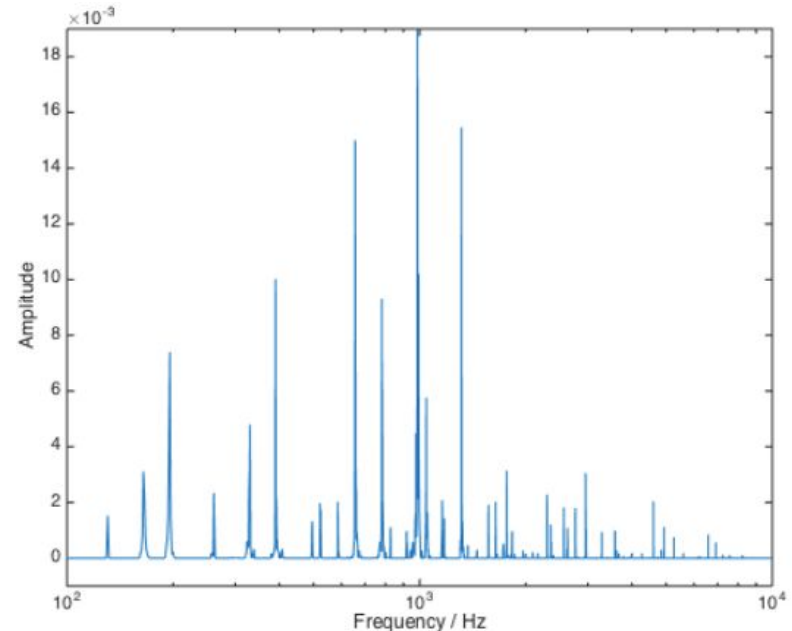
Challenges

- Physical Space to Implement Circuitry was Small; drove us to try wall power instead of Lipo battery; wall power generated too much noise over speaker
- Weird ESP32 Errors (Exit status 2)
- Hard to hide wires for LEDs without lowering structural integrity of the neck
- Amplifier attached to guitar would create a feedback loop causing the sound to continuously ring and get louder
- Clear 3D print for LEDs can blend the colors together to make it a little harder to see where or what finger needs to be placed on the light



Desired Improvements

- Low Pass filter on analog signals from ESP32 to Teensy to eliminate PWM noise and control all digital effect parameters from the app
- Improve tuner code to consistently show the correct frequency and note
- Implement note and chord detection
- Add a song library with chords and melodies



<https://pages.cs.wisc.edu/~kma/downloads/chord-recognition.pdf>